Overriding e funzioni virtuali

Domande d'esame avanzate

C · C++ · Java · Python

Appunti di Ingegneria del Software

31 domande con soluzione ragionata

"All'esame chiede piu' di quello che c'e' scritto:
soprattutto le ***differenze*** *tra i linguaggi."*

Anno 3

Contents

Chapter 1 Come usare questo fascicolo — Page 2
1.1 Fonti segnalate dal docente 2

Chapter 2 C++ — Page 3
2.1 Ereditarieta' virtuale 5

Chapter 3 Java — Page 7

Chapter 4 Python — Page 9

Chapter 5 C — Page 11

Chapter 6 Domande trasversali — Page 13

Chapter 7 Soluzioni — Page 14
7.1 Soluzioni C++ 14
7.2 Soluzioni Java 18
7.3 Soluzioni Python 20
7.4 Soluzioni C 22
7.5 Soluzioni trasversali 24

Chapter 8 Ripasso lampo — Page 26

Chapter 1

Come usare questo fascicolo

Questo fascicolo raccoglie **31 domande di difficolta' alta** su *overriding* e *dispatch dinamico*, pensate per il tipo di interrogazione in cui il professore parte da uno snippet di codice e chiede *"cosa stampa e perche"'*, oppure chiede di confrontare la stessa feature nei quattro linguaggi del corso.
La struttura e' la seguente:

- i capitoli 2–6 contengono *solo le domande*, raggruppate per linguaggio (C++, Java, Python, C) piu' un gruppo trasversale: si possono usare come simulazione a libro chiuso;
- il capitolo 7 contiene le *soluzioni ragionate*, nello stesso ordine; ogni domanda riporta in basso a destra il numero di pagina della propria soluzione.

I codici delle domande (`A1`, `B3`, `C5`, ...) sono stabili: la lettera indica il gruppo, il numero la posizione.

Note:

Tre idee fanno da filo conduttore a quasi tutte le domande, e conviene tenerle a mente perche' sono quelle su cui si sbaglia: **(1)** *cosa* viene deciso a tempo di compilazione (overload, campi, argomenti di default, controllo di accesso) e cosa a tempo di esecuzione (solo il *corpo* del metodo virtuale); **(2)** *quando* il tipo dinamico di un oggetto e' gia' quello finale (in Java sempre, in C++ no durante costruzione e distruzione); **(3)** *dove* avviene la ricerca del metodo (vtable a indice fisso in C++/Java, dizionario lungo l'MRO in Python, struct di puntatori a funzione scritta a mano in C).

1.1 Fonti segnalate dal docente

Il docente ha indicato come riferimento due voci di Wikipedia:

- **Virtual inheritance** — `en.wikipedia.org/wiki/Virtual_inheritance`
  il problema del diamante con l'esempio *Animal / Mammal / WingedAnimal / Bat*, la regola di inizializzazione della base virtuale e il costo in memoria. Corrisponde alle domande A4 e A9–A12.
- **Virtual function** — `en.wikipedia.org/wiki/Virtual_function`
  dispatch dinamico e late binding, metodi virtuali puri e classi astratte, distruttori virtuali, chiamate virtuali durante la costruzione, vtable e relativo overhead, quali linguaggi hanno i metodi virtuali per default. Corrisponde alle domande A1, A8, A11, D1, E1 ed E2.

Le due voci sono il minimo sindacale: quasi tutte le domande di questo fascicolo partono da li' e chiedono il passo successivo, cioe' *perche'* il meccanismo e' fatto cosi' e *come cambia* negli altri tre linguaggi.

Chapter 2

C++

A1 • Chiamate virtuali in costruttore e distruttore

Cosa stampa il programma? Motiva riga per riga.

```
struct A {
    A() { std::cout << "A() "; f(); }
    virtual ~A() { g(); }
    virtual void f() { std::cout << "A::f "; }
    virtual void g() { std::cout << "A::g "; }
};
struct B : A {
    int x = 42;
    void f() override { std::cout << "B::f " << x << " "; }
    void g() override { std::cout << "B::g "; }
};
int main() { B b; }
```

E se in `A` avessi scritto `virtual void f() = 0;`? Cosa cambia, e perche' e' *undefined behavior* e non un errore di compilazione?

Soluzione a pag. 14

A2 • Argomenti di default e binding

Cosa stampa e perche'?

```
struct Base { virtual void f(int x = 10) { std::cout << "Base " << x; } };
struct Der : Base { void f(int x = 20) override { std::cout << "Der " << x; } };
int main() { Base* p = new Der; p->f(); }
```

Soluzione a pag. 14

A3 • Name hiding degli overload

Perche' questo codice compila ma chiama la funzione "sbagliata"? Correggilo in almeno due modi diversi.

```
struct B { virtual void f(int);  void f(double); };
struct D : B { void f(int) override; };
D d;  d.f(3.14);
```

Soluzione a pag. 14

A4 • Dominanza ed ereditarieta' virtuale

Cosa stampa?

```
struct A { virtual void f() { std::cout << "A"; } };
struct B : virtual A { void f() override { std::cout << "B"; } };
struct C : virtual A { };
struct D : B, C { };
int main() { D d; d.f(); }
```

Cosa succederebbe *senza* la parola chiave `virtual` nelle basi? E se anche `C` facesse l'override di `f()`, in quale momento esatto il programma diventa ill-formed?

Soluzione a pag. 15

A5 • Ereditarieta' multipla, thunk e aggiustamento di this

```
struct A { virtual void f(); int a; };
struct B { virtual void g(); int b; };
struct C : A, B { void f() override; void g() override; };
C c;  B* pb = &c;      // (void*)pb != (void*)&c
pb->g();
```

Il `this` che riceve `C::g` deve puntare all'inizio di `C`, ma `pb` punta al sotto-oggetto `B`. Chi esegue l'aggiustamento e dove e' memorizzato? Quanti vptr contiene `c`? Come recuperi l'indirizzo dell'oggetto *most derived* partendo da `pb`?

Soluzione a pag. 15

A6 • Quando una firma e' davvero un override

Quali dei seguenti sono override legali di `virtual void f(int) const noexcept;`? Motiva *ciascuno*.

(a) `void f(int)`	(e) `void f(int) const noexcept &&`
(b) `void f(int) const`	(f) `int f(int) const noexcept`
(c) `void f(int) const noexcept`	(g) `void f(int) const noexcept override final`
(d) `void f(long) const noexcept`	

Soluzione a pag. 15

A7 • Private virtual e pattern NVI

Spiega il pattern NVI (*Non-Virtual Interface*) e di' cosa succede a ciascuna delle tre righe finali.

```
class B {
public:  void call() { f(); }
private: virtual void f() { std::cout << "B"; }
};
class D : public B {
public: void f() override { std::cout << "D"; }
};
B* p = new D;
p->call();                    // (1)
p->f();                       // (2)
static_cast<D*>(p)->f();      // (3)
```

Perche' un metodo `private virtual` e' comunque overridabile? Su quale tipo viene fatto il controllo di accesso?

Soluzione a pag. 15

A8 • Cinque "perche"' sul modello a oggetti del C++

Rispondi con motivazione tecnica (non "perche' lo dice lo standard"):

(a) perche' un **costruttore** non puo' essere `virtual`, e come si implementa un `clone()` polimorfico con return covariante?

(b) perche' una **member function template** non puo' essere `virtual`?

(c) una funzione virtuale puo' essere `inline`? Con quale effetto reale?

(d) cosa permette al compilatore `final` che `override` non permette?

(e) perche' `delete[] pBase` su un array di `Derived` e' UB *anche* con distruttore virtuale?

Soluzione a pag. 16

2.1 Ereditarieta' virtuale

A9 • Chi costruisce la base virtuale

Cosa stampa? E cosa stamperebbe `D d;` al posto di `E e;` ? E `B b;` ?

```
struct A { A(const char* who) { std::cout << who << " "; } };
struct B : virtual A { B() : A("da B") {} };
struct C : virtual A { C() : A("da C") {} };
struct D : B, C  { D() : A("da D"), B(), C() {} };
struct E : D     { E() : A("da E"), D()      {} };
int main() { E e; }
```

Cosa succede se togli `A("da E")` dalla lista di inizializzazione di `E`? E qual e' l'ordine di costruzione dei sotto-oggetti?

Soluzione a pag. 16

A10 • Layout e costo della base virtuale

Nel classico esempio `Animal` / `Mammal` / `WingedAnimal` / `Bat` con ereditarieta' virtuale, il layout di `Bat` diventa

`(vpointer, Mammal, vpointer, WingedAnimal, Bat, Animal)`

Rispondi: perche' il sotto-oggetto `Animal` finisce **in fondo** e non all'inizio? Perche' servono **due** vpointer? Perche' l'offset fra un sotto-oggetto `Mammal` e il suo `Animal` non e' una costante di compilazione, come invece succede nell'ereditarieta' non virtuale? E infine: quale delle due righe seguenti e' ill-formed, e perche' ?

```
Animal* pa = &bat;
Bat* p1 = static_cast<Bat*>(pa);
Bat* p2 = dynamic_cast<Bat*>(pa);
```

Soluzione a pag. 16

A11 • Metodi virtuali puri nel diamante

```
struct I { virtual void f() = 0; virtual ~I() = default; };
struct L : virtual I { void f() override { std::cout << "L"; } };
struct R : virtual I { };
struct D : L, R { };
int main() { D d; d.f(); }
```

`D` e' istanziabile? Cosa cambia se togli `virtual` dalle basi di `L` e `R`? Infine: un metodo virtuale *puro* puo'

avere un **corpo**? Se si', come lo si chiama e a cosa serve? In quale caso quel corpo e' *obbligatorio*?

Soluzione a pag. 17

A12 • Quando usarla, e come se la cavano gli altri tre linguaggi

Il criterio classico e' che l'ereditarieta' virtuale serve quando l'ereditarieta' esprime una *restrizione di un insieme* e non una *composizione di parti*. Spiega il criterio, poi rispondi:

- in una frase: che differenza c'e', a livello di rappresentazione in memoria, tra base virtuale e base non virtuale?
- perche' in **Java** il problema del diamante *sui dati* non puo' nemmeno presentarsi, e quale diamante resta?
- perche' in **Python** si potrebbe dire che l'ereditarieta' e' "sempre virtuale"?
- come si sceglie fra le due in **C**?

Cita infine un esempio reale di ereditarieta' virtuale nella libreria standard del C++.

Soluzione a pag. 17

Chapter 3

Java

B1 • Campi nascosti contro metodi sovrascritti

Cosa stampa? Ci sono tre risultati diversi: spiegali tutti e tre.

```
class A { String x = "A"; String get() { return x; } String getX() { return x; } }
class B extends A { String x = "B"; String get() { return x; } }

A a = new B();
System.out.println(a.x);         // (1)
System.out.println(a.get());     // (2)
System.out.println(a.getX());    // (3)
System.out.println(((B)a).x);    // (4)
```

Soluzione a pag. 18

B2 • Metodi statici e istanze

Cosa stampa e perche'? Come si chiama tecnicamente il fenomeno (non e' overriding)?

```
class A { static String s() { return "A"; } }
class B extends A { static String s() { return "B"; } }
A a = new B();
System.out.println(a.s());
```

Soluzione a pag. 18

B3 • Metodo overridabile chiamato dal costruttore

Cosa stampa? Descrivi l'ordine *esatto* di inizializzazione.

```
class A { A() { init(); } void init() { } }
class B extends A {
    final int x;
    List<String> list = new ArrayList<>();
    B() { x = 5; }
    @Override void init() { System.out.println(x); System.out.println(list); }
}
new B();
```

Soluzione a pag. 18

B4 • Overriding o overloading?

Cosa stampa ciascuna riga?

```java
class A { void m(Object o) { System.out.println("A/Object"); } }
class B extends A { void m(String s) { System.out.println("B/String"); } }

A a = new B();  B b = new B();
a.m("hi");            // (1)
b.m("hi");            // (2)
b.m((Object)"hi");    // (3)
```

Soluzione a pag. 19

B5 • Generics, erasure e bridge method

Perche' questo lancia `ClassCastException` e, soprattutto, *in quale metodo* viene lanciata? Che bytecode ha generato il compilatore?

```java
class Node<T> { T data; void set(T t) { this.data = t; } }
class MyNode extends Node<Integer> { @Override void set(Integer i) { super.set(i); } }

Node n = new MyNode();
n.set("stringa");
```

Soluzione a pag. 19

B6 • Diamante di default method

Perche' non compila? Scrivi la correzione.

```java
interface A { default String f() { return "A"; } }
interface B extends A { default String f() { return "B"; } }
interface C extends A { default String f() { return "C"; } }
class D implements B, C { }
```

Poi spiega: (a) la regola *class wins*; (b) perche' un'interfaccia **non puo'** dichiarare `default String toString()`; (c) perche' esiste `invokeinterface` distinto da `invokevirtual`.

Soluzione a pag. 19

B7 • Le regole dell'override in Java

Quali di queste operazioni sono legali? Motiva.

(a) restringere la visibilita' `public` → `protected`;

(b) allargarla `protected` → `public`;

(c) return covariante;

(d) aggiungere una *checked exception* non dichiarata nella base;

(e) fare l'override di un metodo `private` della superclasse;

(f) un metodo `static` che fa l'override di uno di istanza;

(g) scrivere `equals(MyType)` invece di `equals(Object)`: che bug produce con `HashSet.contains()`?

Soluzione a pag. 19

Chapter 4

Python

C1 • MRO, C3 e super() cooperativo

Dai l'MRO di `D` e l'output. Perche' `super()` dentro `B` *non* chiama `A.f`?

```
class A:
    def f(self): print("A")
class B(A):
    def f(self): print("B"); super().f()
class C(A):
    def f(self): print("C"); super().f()
class D(B, C):
    def f(self): print("D"); super().f()
D().f()
```

Poi: perche' `class X(A, B)` con `class B(A)` solleva `TypeError`?

Soluzione a pag. 20

C2 • Lookup implicito dei metodi speciali

Cosa stampano le due righe? Enuncia la regola.

```
class A:
    def __len__(self): return 1
a = A()
a.__len__ = lambda: 99
print(len(a))
print(a.__len__())
```

Perche' questo rende difficile scrivere un *proxy* trasparente basato su `__getattr__`?

Soluzione a pag. 20

C3 • Name mangling

Cosa stampa? Come si chiama il meccanismo e come lo si usa per ottenere l'equivalente di un metodo *non* virtuale?

```
class A:
    def __init__(self): self.__helper()
    def __helper(self): print("A helper")
class B(A):
    def __helper(self): print("B helper")
B()
```

Soluzione a pag. 20

C4 • Dove vive davvero un metodo

Analizza riga per riga.

```
class A:
    def f(self): print("A.f")
class B(A):
    def f(self): print("B.f")

b = B()
b.f()                                   # (1)
A.f(b)                                  # (2)
B.f = A.f;  b.f()                       # (3)
b.f = lambda: print("inst");  b.f()     # (4)
del b.f
b.__class__ = A;  b.f()                 # (5)
```

Perche' al punto (4) l'attributo di istanza vince sul metodo di classe, mentre con una `property` non succederebbe?

Soluzione a pag. 20

C5 • Priorita' delle operazioni riflesse

Cosa stampa e perche'? Qual e' la regola precisa di CPython?

```
class A:
    def __add__(self, other): return "A.__add__"
class B(A):
    def __radd__(self, other): return "B.__radd__"
print(A() + B())
```

Soluzione a pag. 21

C6 • Il costruttore virtuale che il C++ non ha

Cosa stampa? Perche' questo e' il modo idiomatico di scrivere factory polimorfiche?

```
class A:
    @classmethod
    def create(cls, *args): return cls(*args)
    def __repr__(self): return type(self).__name__
class B(A): pass
print(B.create())
```

Cosa cambierebbe con uno `@staticmethod` che restituisce `A()`?

Soluzione a pag. 21

C7 • Metodi astratti che non astraggono

Perche' questo **non** solleva alcun errore all'istanziazione, e come si corregge?

```
from abc import abstractmethod
class Base:
    @abstractmethod
    def f(self): ...
class Impl(Base): pass
Impl()
```

Bonus: ai fini dell'overriding, che differenza c'e' tra `__init_subclass__`, una metaclasse e `ABCMeta.__subclasshook__`?

Soluzione a pag. 21

Chapter 5

C

D1 • Dispatch dinamico scritto a mano

Implementa il dispatch dinamico in C puro: una `struct Shape` con vtable, un `Circle` che "eredita", e la funzione `draw_all`. Poi rispondi:

- perche' il sotto-oggetto base deve essere il **primo** membro?
- chi inizializza il vptr, e cosa succede se te lo dimentichi (confronto con il C++)?
- come emuli un **distruttore virtuale** e un `dynamic_cast`?

Soluzione a pag. 22

D2 • Il cast che sembra innocuo

Perche' questo e' formalmente scorretto in C, anche se "funziona" su tutti i compilatori?

```
void draw_all(Shape **v, size_t n);
Circle *arr[3] = { c1, c2, c3 };
draw_all((Shape **)arr, 3);
```

Cosa dicono lo standard sulla *common initial sequence* e la regola di *strict aliasing*? Perche' la garanzia "indirizzo del primo membro = indirizzo della struct" non basta a salvarti qui?

Soluzione a pag. 22

D3 • Traduzione letterale di virtual

In C++ scrivi `virtual double area() const;`. Traduci *letteralmente* la sua rappresentazione a runtime: cosa diventa `this`, cosa diventa `const`, cos'e' la vtable e come viene "ereditata" dalla classe derivata. Qual e' il costo in byte per oggetto e per classe?

Soluzione a pag. 23

D4 • Due polimorfismi diversi nella libreria standard e nel kernel

`struct file_operations` del kernel Linux e il comparatore di `qsort(..., int (*cmp)(const void*, const void*))` sono due forme diverse di polimorfismo. Quale corrisponde all'overriding e quale alla *strategy*/callback? Cosa fa la macro `container_of` e a quale operazione del C++ corrisponde?

Soluzione a pag. 23

D5 • Perche' il C non puo' avere overriding automatico

Perche' in C non puoi ottenere overriding "automatico" nemmeno usando `static inline` negli header? Collega la risposta a: *early binding*, unita' di traduzione, assenza di *name mangling* e assenza di costruttori.

Soluzione a pag. 23

Chapter 6

Domande trasversali

E1 • Tabella comparativa del dispatch

Compila la tabella per **C++, Java, Python, C**:

(i) meccanismo di dispatch a runtime;

(ii) costo per chiamata;

(iii) e' possibile aggiungere o rimpiazzare un metodo a runtime?

(iv) esiste il *method hiding*?

(v) come si chiama esplicitamente l'implementazione della base?

Soluzione a pag. 24

E2 • Virtuale per default oppure no

In C++ le funzioni sono non virtuali per default, in Java sono virtuali per default (salvo `final`, `private`, `static`), in Python tutto e' dinamico. Argomenta il trade-off usando: *zero-overhead principle*, *fragile base class problem* e devirtualizzazione (CHA + inline caching nella JVM contro LTO e `final` in C++).

Soluzione a pag. 24

E3 • Covarianza e controvarianza

Data `Base` con `void process(Base b)` e `Derived` con `void process(Derived d)`: in quale dei quattro linguaggi questo e' un override? Collega la risposta alla **covarianza/controvarianza dei parametri** e al principio di sostituzione di Liskov. Perche' nessun linguaggio mainstream ammette parametri controvarianti?

Soluzione a pag. 24

E4 • Sette termini da non confondere

Definisci con precisione e distingui: *overriding*, *overloading*, *hiding/shadowing*, *dominanza*, *bridge method*, *thunk*, *final overrider*. Per ciascuno indica in quale dei quattro linguaggi esiste.

Soluzione a pag. 25

Chapter 7

Soluzioni

7.1 Soluzioni C++

Soluzione A1 — Chiamate virtuali in costruttore e distruttore

Output: `A() A::f A::g`.
Durante il corpo del costruttore di `A` il tipo dinamico dell'oggetto **e'** `A`: il vptr e' stato impostato a quello di `A` e verra' riscritto solo all'inizio del costruttore di `B`. Quindi `f()` risolve ad `A::f` — e per fortuna, perche' `B::x` non e' ancora inizializzato e `B::f` lo leggerebbe.
Simmetricamente `~A()` gira *dopo* `~B()`: quando si entra nel corpo di `~A` il vptr e' gia' stato riportato ad `A`, quindi `g()` chiama `A::g`.
Regola: mai chiamare funzioni virtuali da costruttore o distruttore aspettandosi dispatch dinamico. Il tipo dinamico "cresce" durante la costruzione e "decresce" durante la distruzione.
Con `f()` pura, la chiamata dal costruttore di `A` e' **undefined behavior** (*[class.abstract]*). Non e' un errore di compilazione perche' la chiamata e' sintatticamente lecita e in generale il compilatore non puo' dimostrare che verra' eseguita (potrebbe essere dentro un `if`, o in una funzione chiamata dal costruttore). In pratica quasi tutte le implementazioni mettono nella vtable di `A` il simbolo `__cxa_pure_virtual`, che termina il programma con il messaggio "pure virtual method called".

Soluzione A2 — Argomenti di default e binding

Stampa `Der 10`.
Il *corpo* della funzione e' scelto dinamicamente (tramite vtable), ma gli **argomenti di default sono sostituiti a tempo di compilazione** in base al *tipo statico* dell'espressione, che qui e' `Base*`. Il compilatore genera di fatto `p->f(10)` e poi il dispatch porta a `Der::f`.
Da qui la regola pratica: **mai ridefinire gli argomenti di default in un override**. Se servono default diversi, si usa NVI (vedi A7): la funzione pubblica non virtuale porta il default, quella virtuale privata non ne ha.

Soluzione A3 — Name hiding degli overload

`D::f(int)` **nasconde** tutti gli overload `B::f`: il *name lookup* risale gli scope e si ferma al *primo* che contiene il nome `f`, cioe' `D`. Solo dopo, sull'insieme trovato, parte l'*overload resolution*. Quindi l'unico candidato e' `D::f(int)`, e `3.14` viene convertito a `int` (con troncamento silenzioso).
Correzioni possibili:

```
struct D : B {
    using B::f;              // (1) reintroduce tutti gli overload della base
    void f(int) override;
};
d.B::f(3.14);                // (2) chiamata qualificata
// (3) ridichiarare esplicitamente void f(double) in D
```

Il flag `-Woverloaded-virtual` di GCC/Clang intercetta il caso.

Soluzione A4 — Dominanza ed ereditarieta' virtuale

Stampa `B`.
Con basi virtuali esiste **un solo** sotto-oggetto `A` condiviso, e la regola di **dominanza** stabilisce che `B::f` domina `A::f` (perche' `B` deriva da `A`). Il *final overrider* di `A::f` in `D` e' quindi unico: nessuna ambiguita'.
Senza `virtual` nelle basi ci sarebbero **due** sotto-oggetti `A` distinti, quindi due `A::f` indipendenti: `d.f()` e' ambiguo (errore di compilazione) e servirebbe `d.B::f()`.
Se anche `C` facesse l'override, non esisterebbe piu' un final overrider unico: il programma e' **ill-formed alla definizione di** `D` (non al punto di chiamata), a meno che `D` non fornisca il proprio override, che allora domina entrambi.

Soluzione A5 — Ereditarieta' multipla, thunk e aggiustamento di this

L'aggiustamento del `this` lo esegue un **thunk**: nella *vtable secondaria* (quella associata al sotto-oggetto `B` dentro `C`) la entry per `g` non punta a `C::g`, ma a un piccolo trampolino che fa `this -= offsetof(C, B-subobject)` e poi salta a `C::g`. Nell'Itanium ABI si chiama *non-virtual this adjustment*; altre ABI memorizzano l'offset accanto al puntatore a funzione.
`c` contiene **due vptr**: uno per il sotto-oggetto primario (`A`) e uno per `B`. Ecco perche' `sizeof(C)` e' maggiore di quanto ci si aspetterebbe dai soli campi.
L'indirizzo dell'oggetto *most derived* si ottiene con `dynamic_cast<void*>(pb)`, che usa l'*offset-to-top* memorizzato nella vtable prima delle entry dei metodi.

Soluzione A6 — Quando una firma e' davvero un override

(a) `void f(int)` — **no**: perde `const`, quindi la firma e' diversa. Diventa un overload che *nasconde* il virtuale; con `override` il compilatore da' errore.

(b) `void f(int) const` — **no**: da C++17 `noexcept` fa parte del *tipo* della funzione, e un override non puo' essere *meno* restrittivo della base. (Il contrario — base non `noexcept`, derivata `noexcept` — e' invece legale.)

(c) `void f(int) const noexcept` — **si'**, e' l'override corretto.

(d) `void f(long) const noexcept` — **no**: parametri diversi, quindi overload + hiding.

(e) con `&&` — **no**: i *ref-qualifier* fanno parte della firma e devono coincidere esattamente.

(f) `int f(int) const noexcept` — **no**: il return type e' diverso e non covariante. La covarianza vale solo per puntatori o riferimenti a classi in relazione di derivazione (e non per `int` contro `void`).

(g) `override final` in una classe gia' `final` — legale, ma ridondante.

Soluzione A7 — Private virtual e pattern NVI

(1) stampa `D`; (2) **errore di compilazione**; (3) compila e stampa `D`.
Il *name lookup* e il *controllo di accesso* avvengono sul **tipo statico** e sono completamente ortogonali al dispatch dinamico, che avviene sul tipo dinamico. Per questo un `private virtual` e' perfettamente overridabile: la classe derivata non ha bisogno di *chiamarlo*, solo di *ridefinirlo*.
NVI: l'interfaccia pubblica e' non virtuale (`call()`) e i punti di variazione sono virtuali privati. In questo modo la base mantiene il controllo su pre/post-condizioni, logging, lock, invarianti — ed e' libera di cambiare il numero e la forma dei punti di estensione senza rompere le derivate. Corollario noto (Sutter): in una classe base pubblica, l'unica funzione che dovrebbe essere `public` *e* `virtual` e' il distruttore.

Soluzione A8 — Cinque "perche"' sul modello a oggetti del C++

(a) Il dispatch virtuale richiede un oggetto gia' costruito con un vptr valido, ed e' esattamente il costruttore a installarlo: un costruttore virtuale sarebbe circolare. L'idioma sostitutivo e' il *virtual constructor* di Coplien:

```
struct Base {
    virtual Base* clone() const { return new Base(*this); }
    virtual ~Base() = default;
};
struct Der : Base {
    Der* clone() const override { return new Der(*this); }  // return covariante
};
```

(b) Una member template genera un numero potenzialmente illimitato di istanziazioni, scoperte in unita' di traduzione diverse e anche molto dopo la definizione della classe: la vtable dovrebbe avere dimensione e layout non determinabili nel punto in cui la classe viene definita. (Una *class* template puo' invece avere tutte le funzioni virtuali che vuole: per ogni istanziazione concreta l'insieme dei metodi e' noto.)
(c) Si'. `inline` resta utile per le chiamate risolte staticamente (`obj.f()` su un oggetto, `Base::f()` qualificato, o dopo devirtualizzazione); il compilatore deve comunque emettere anche una copia *out-of-line*, perche' l'indirizzo della funzione finisce nella vtable.
(d) `override` e' solo un *controllo*: "verifica che io stia davvero sovrascrivendo qualcosa". `final` e' invece un'*informazione semantica* — nessuno potra' piu' fare l'override — che permette al compilatore di **devirtualizzare** la chiamata e quindi di inlinearla, anche senza LTO o analisi dell'intero programma.
(e) L'aritmetica dei puntatori negli array usa `sizeof` del **tipo statico**: `pBase[1]` verrebbe calcolato con passo `sizeof(Base)` invece di `sizeof(Derived)`. Il distruttore virtuale non c'entra nulla: il problema e' l'indicizzazione, non la distruzione. La soluzione e' `std::vector<std::unique_ptr<Base>>`.

Soluzione A9 — Chi costruisce la base virtuale

Stampa `da E`, **una volta sola**.
La regola e' che **la base virtuale viene inizializzata dal costruttore della classe *most derived***, cioe' dalla classe dell'oggetto che si sta effettivamente creando. Tutti gli altri mem-initializer per `A` — quelli in `B`, `C` e `D` — vengono semplicemente *ignorati*, perche' in questa costruzione quelle classi sono sotto-oggetti, non l'oggetto piu' derivato.
Quindi: `D d;` stampa `da D`, e `B b;` stampa `da B`. Lo stesso mem-initializer `A("da B")` viene usato o ignorato a seconda del ruolo che `B` ha in quella particolare costruzione: ecco perche' scriverlo in `B` non e' inutile, serve per il caso in cui `B` sia standalone.
Se togli `A("da E")` da `E`, il compilatore deve inizializzare `A` per default — ma `A` non ha costruttore di default, quindi il programma e' **ill-formed**. Conseguenza pratica: una base virtuale conviene che abbia un costruttore di default, altrimenti *ogni* classe concreta della gerarchia (a qualunque profondita') e' obbligata a nominarla esplicitamente. E' uno dei costi "sociali" dell'ereditarieta' virtuale.
Ordine di costruzione: prima tutte le *basi virtuali*, in ordine di visita depth-first da sinistra a destra della gerarchia della classe most derived; poi le basi non virtuali nell'ordine di dichiarazione; poi i membri nell'ordine di dichiarazione; infine il corpo del costruttore. Qui: `A`, `B`, `C`, `D`, `E`. La distruzione avviene in ordine esattamente inverso.

Soluzione A10 — Layout e costo della base virtuale

`Animal` finisce **in fondo** proprio perche' e' *condiviso*: i sotto-oggetti `Mammal` e `WingedAnimal` devono conservare il layout che avrebbero da soli (cosi' il codice compilato per `Mammal` continua a funzionare), quindi l'unica copia della base comune viene messa in una posizione che non appartiene a nessuno dei due rami. Nel caso non virtuale, invece, ogni ramo si porta dietro la *propria* copia della base, incorporata al proprio inizio.
Servono **due vpointer** perche' ciascun ramo che eredita virtualmente ha bisogno di sapere "quanto dista da qui il mio `Animal`", e questa distanza e' diversa a seconda di come il sotto-oggetto e' immerso nell'oggetto completo. L'informazione si chiama **vbase offset** e sta nella vtable, accanto ai puntatori ai metodi.
Perche' non e' una costante di compilazione? Perche' quando il compilatore traduce un metodo di `Mammal`

non sa se quel `Mammal` sara' un oggetto autonomo (e allora `Animal` sta subito dopo il vptr) oppure il sotto-oggetto di un `Bat` (e allora `Animal` sta molto piu' in la', dopo `WingedAnimal` e dopo i membri di `Bat`). Con ereditarieta' non virtuale l'offset base/derivata e' invece fissato una volta per tutte a compile time — ed e' esattamente la differenza di costo fra i due meccanismi: nel caso virtuale l'accesso a un membro della base costa un'indirezione in piu'. Nel conto di Wikipedia, la dimensione dell'oggetto `Bat` cresce di due puntatori rispetto alla versione non virtuale, in cambio dell'unicita' di `Animal` e dell'assenza di ambiguita'.
Il cast: e' ill-formed la riga con `static_cast`. Un `static_cast` da base virtuale a derivata e' *vietato dallo standard*, e per il motivo appena detto: il compilatore dovrebbe sottrarre un offset che non conosce staticamente. Serve `dynamic_cast`, che legge l'informazione dalla vtable a runtime (ed e' anche l'unico caso in cui `dynamic_cast` e' obbligatorio pur non essendoci alcuna incertezza sul tipo). Per lo stesso motivo non si puo' usare `offsetof` su un membro della base virtuale, e i puntatori a membro della base virtuale hanno una rappresentazione piu' complessa.

Soluzione A11 — Metodi virtuali puri nel diamante

Si', `D` e' istanziabile e stampa `L`. Con le basi virtuali esiste un solo sotto-oggetto `I`, e per la regola di dominanza (A4) il suo *final overrider* unico e' `L::f`. Il ramo `R` non ha bisogno di implementare nulla: eredita l'implementazione "dall'altra parte" del diamante. E' esattamente il punto sottolineato da Wikipedia — con ereditarieta' virtuale il metodo puro va implementato *una volta sola* nella gerarchia.
Togliendo `virtual` dalle basi, ci sono **due** sotto-oggetti `I` indipendenti: quello raggiunto via `R` ha ancora `f` pura, quindi `D` resta **astratta** e `D d;` non compila. E anche se implementassi `f` in `R`, la chiamata `d.f()` sarebbe ambigua e servirebbe `d.L::f()`.
Un metodo puro puo' avere un corpo. `= 0` dichiara solo che le derivate concrete devono fornire un override; non vieta di dare un'implementazione di *default*:

```
struct I { virtual void f() = 0; };
void I::f() { std::cout << "comportamento di base"; }
struct D : I { void f() override { I::f(); std::cout << " + extra"; } };
```

Il corpo si puo' invocare solo con **chiamata qualificata** (`I::f()`), mai per dispatch dinamico: la entry in vtable resta quella "pura". Serve come fallback riusabile dalle derivate — ed e' lo stesso schema di Python, dove un `@abstractmethod` puo' avere un corpo che la sottoclasse richiama con `super()`.
Il corpo e' **obbligatorio** in un caso: il *distruttore virtuale puro*. `virtual ~I() = 0;` rende la classe astratta (idioma utile quando non ci sono altri metodi da rendere puri), ma il distruttore di ogni derivata chiama comunque quello della base, quindi la definizione deve esistere: `I::~I() {}`.

Soluzione A12 — Quando usarla, e come se la cavano gli altri tre linguaggi

Il criterio. Se l'ereditarieta' esprime una *restrizione di un insieme* — un `Bat` *e'* un `Animal`, e ne esiste uno solo per ogni pipistrello — allora la base va condivisa, quindi `virtual`. Se invece l'ereditarieta' viene usata per *comporre parti* indipendenti (un `Bat` "contiene" le caratteristiche del volo e quelle dei mammiferi come due moduli distinti), avere due copie e' coerente, e la base non virtuale va bene. In pratica: la duplicazione e' un bug quando la base rappresenta *identita'*, e' accettabile quando rappresenta *implementazione*.
Rappresentazione in memoria, in una frase: una base non virtuale e' *incorporata* (una copia per ogni ramo, offset costante), una base virtuale e' *condivisa* (una copia sola, raggiunta tramite un offset letto a runtime) — cioe' concettualmente "per valore" contro "per riferimento".
Java. Una classe puo' estendere una sola classe, quindi lo stato non puo' arrivare da due strade: il diamante *sui dati* e' impedito per costruzione. Resta il diamante *sul comportamento*, quello dei `default method` delle interfacce (B6), che il linguaggio non risolve automaticamente ma obbliga a risolvere a mano con un override esplicito e `X.super.f()`. Le interfacce sono in un certo senso "sempre virtuali": non portando stato, non c'e' niente da duplicare.
Python. L'ereditarieta' multipla di stato e' permessa, ma gli attributi vivono in *un solo* `__dict__` per istanza, e la linearizzazione C3 mette ogni classe base *una volta sola* nell'MRO. Quindi Python si comporta di fatto come se ogni base fosse virtuale: c'e' una sola `A`, e se tutti cooperano con `super()` il suo `__init__` viene eseguito esattamente una volta (C1). Il prezzo lo si paga altrove: l'ordine di inizializzazione dipende

dall'MRO della classe concreta, esattamente come in C++ la base virtuale e' costruita dalla most derived (A9).
C. La scelta e' esplicita e si vede: base non virtuale = `struct A base;` incorporata come primo membro (D1); base virtuale = `struct A *base;`, un puntatore alla copia condivisa. Il costo dell'ereditarieta' virtuale in C++ e' letteralmente il costo di quella sostituzione — un'indirezione in piu' e un puntatore in piu' per ramo.
Esempio reale nella libreria standard: la gerarchia degli stream. `basic_istream` e `basic_ostream` derivano *virtualmente* da `basic_ios`, cosicche' `basic_iostream` (da cui `std::stringstream`, `std::fstream`) abbia un solo stato di stream — un solo buffer, un solo insieme di flag di errore — invece di due. Se l'ereditarieta' non fosse virtuale, `ss.fail()` potrebbe dare risposte diverse a seconda del ramo da cui la si interroga.

7.2 Soluzioni Java

Soluzione B1 — Campi nascosti contro metodi sovrascritti

(1) `A` — i **campi non sono polimorfici**: l'accesso e' risolto a tempo di compilazione sul tipo statico della variabile (`A a`), con l'istruzione `getfield A.x`.

(2) `B` — i metodi si': `invokevirtual` risolve sul tipo dinamico.

(3) `A` — `getX()` e' definito *dentro* `A`, e nel suo corpo l'identificatore `x` si riferisce ad `A.x`.

(4) `B`.

Il punto chiave e' che `B.x` non *sostituisce* `A.x`: lo **nasconde**. Nell'oggetto esistono *due* campi distinti, e `A.x` resta raggiungibile da dentro `B` con `super.x` oppure `((A)this).x`. Morale: i campi si dichiarano `private` e si espongono con metodi, altrimenti l'ereditarieta' diventa illeggibile.

Soluzione B2 — Metodi statici e istanze

Stampa `A`. E' **hiding di metodo statico**, non overriding: la chiamata viene compilata in `invokestatic A.s()` sulla base del tipo statico di `a`; l'oggetto non viene nemmeno usato (`javac` emette il warning "static method should be qualified by type name").
I metodi `static` sono legati staticamente *per definizione*: non esiste un ricevitore su cui fare dispatch, quindi non esiste nemmeno un slot in vtable. Lo stesso vale per `private` (`invokespecial`) e per i metodi `final`, che pero' uno slot ce l'hanno.

Soluzione B3 — Metodo overridabile chiamato dal costruttore

Stampa `0` e poi `null`.
Ordine di inizializzazione:

1. `super()` — e qui sta il punto: il costruttore di `A` chiama `init()`, che e' **gia'** dispatchato dinamicamente su `B`. A differenza del C++ (A1), in Java il tipo dinamico dell'oggetto e' quello finale *fin dal primo istante*: non esiste il "vptr che cresce";
2. *dopo* il ritorno da `super()` vengono eseguiti gli inizializzatori di campo e i blocchi di istanza di `B`, in ordine testuale;
3. infine il corpo del costruttore di `B`.

Percio' durante `init()` i campi di `B` valgono ancora il default: `x == 0` *anche se dichiarato* ***`final`***, e `list == null` (NullPointerException se ci si fa qualcosa).
E' l'errore classico documentato in *Effective Java* (item 19): **mai invocare metodi overridabili dal costruttore**. Se serve, si usa un metodo `private` o `final`, oppure un factory method che costruisce e *poi* inizializza.

Soluzione B4 — Overriding o overloading?

(1) `A/Object` (2) `B/String` (3) `A/Object`.
`B.m(String)` **non** fa l'override di `A.m(Object)`: le firme sono diverse, quindi e' un *overload* che si aggiunge al metodo ereditato.
La risoluzione dell'overload avviene a **tempo di compilazione** sul tipo statico: con `A a` l'unico candidato visibile e' `m(Object)`; con `B b` i candidati sono due e vince il piu' specifico, `m(String)`; col cast esplicito il tipo statico dell'argomento torna `Object`.
Slogan da ripetere all'esame: *overload = statico, override = dinamico.* Mescolare i due e' la fonte di bug piu' comune sull'argomento (vedi anche B7g con `equals`).

Soluzione B5 — Generics, erasure e bridge method

L'eccezione viene lanciata **dentro il bridge method sintetico** `MyNode.set(Object)`, non nel codice scritto dal programmatore.
Per *erasure*, `Node.set(T)` diventa `set(Object)`. Ma `MyNode.set(Integer)` ha un descriptor diverso, quindi da solo *non* sarebbe un override per la JVM. Il compilatore genera allora in `MyNode`:

```
// ACC_BRIDGE | ACC_SYNTHETIC
void set(Object o) { this.set((Integer) o); }
```

Poiche' `n` e' dichiarata di tipo **raw**, il compilatore non inserisce alcun `checkcast` al call site: viene emessa `invokevirtual Node.set(Object)`, il dispatch arriva al bridge, il bridge esegue `checkcast Integer` su una `String` e lancia `ClassCastException`. Il fenomeno si chiama *heap pollution*, ed e' il prezzo della retrocompatibilita' scelta da Java (erasure) invece della reificazione dei generici (come in C#).
I bridge method vengono generati anche per i **return covarianti**: se la base restituisce `Object` e la derivata `String`, servono due metodi.

Soluzione B6 — Diamante di default method

`D` eredita due default method `f()` egualmente specifici e non correlati fra loro: `B` e `C` sono entrambe sotto-interfacce di `A`, ma nessuna delle due e' sotto-interfaccia dell'altra, quindi non c'e' un criterio per preferirne una. Errore: *inherits unrelated defaults*. Correzione:

```
class D implements B, C {
    @Override public String f() { return B.super.f(); }
}
```

(a) Class wins. Un metodo concreto ereditato da una *superclasse* batte sempre un default method, anche se l'interfaccia sembra "piu' vicina". La regola esiste per garantire la retrocompatibilita': aggiungere un default a un'interfaccia gia' pubblicata non deve mai cambiare il comportamento del codice esistente.
(b) I metodi di `Object` sono ereditati da *ogni* classe: per la regola "class wins" un default `toString()` sarebbe sempre scavalcato e quindi inutile o ambiguo. Il JLS lo vieta esplicitamente a tempo di compilazione.
(c) Con l'ereditarieta' singola delle classi, ogni metodo virtuale ha uno **slot a indice fisso** nella vtable, identico in tutte le sottoclassi: `invokevirtual` e' quindi un accesso a offset costante. Le interfacce invece possono essere implementate da classi con gerarchie completamente diverse, e non esiste un indice comune: `invokeinterface` richiede una ricerca (itable, itable stub, o inline cache costruita dal JIT).

Soluzione B7 — Le regole dell'override in Java

(a) **illegale**: non si puo' restringere la visibilita', o si violerebbe la sostituibilita' (un client che vede `Base` potrebbe chiamare il metodo).

(b) **legale**: allargare si puo'.

(c) **legale** da Java 5 (implementata con bridge method, B5).

(d) **illegale** per le *checked* exception: si possono solo restringere o omettere. Le *unchecked* sono libere,

perche' non fanno parte del contratto verificato dal compilatore.

(e) **impossibile**: i metodi `private` non sono ereditati ne' virtuali (`invokespecial`); definirne uno omonimo nella derivata crea un metodo del tutto indipendente.

(f) **illegale** in entrambe le direzioni: `static` non puo' nascondere un metodo di istanza ne' viceversa.

(g) `equals(MyType)` e' un *overload*. Tutte le collezioni (`HashSet`, `HashMap`, `List.contains`) chiamano `equals(Object)`, che rimane quello di `Object`, cioe' l'identita' di riferimento: `contains` restituisce `false` su oggetti logicamente uguali. Annotare sempre con `@Override` — sulla firma sbagliata da' errore di compilazione — e ricordare che `equals` va sempre accompagnato da `hashCode`.

7.3 Soluzioni Python

Soluzione C1 — MRO, C3 e super() cooperativo

MRO: `D` → `B` → `C` → `A` → `object` (linearizzazione C3). Output: `D B C A`.
Il punto cruciale e' che `super()` **non significa "la superclasse"**, ma "il prossimo nell'MRO *dell'oggetto*". Dentro `B.f`, con `type(self) is D`, il successivo dopo `B` nella lista e' `C`, non `A`. E' esattamente questo che rende possibile l'ereditarieta' multipla *cooperativa*: ogni classe chiama `super()` una volta sola e ogni implementazione di `f` viene eseguita esattamente una volta, in ordine deterministico.
Corollario pratico: in una gerarchia cooperativa tutti i metodi devono avere firme compatibili (tipicamente `*args, **kwargs`), altrimenti la catena si rompe a meta'.
`class X(A, B)` con `class B(A)`: C3 deve rispettare sia l'ordine locale delle basi (`A` prima di `B`) sia la monotonicita' (`B` prima di `A`, perche' `B` deriva da `A`). I due vincoli sono contraddittori, quindi si ottiene `TypeError: Cannot create a consistent method resolution order (MRO)`.

Soluzione C2 — Lookup implicito dei metodi speciali

`len(a)` stampa **1**; `a.__len__()` stampa **99**.
L'invocazione **implicita** dei metodi speciali salta il dizionario dell'istanza e cerca lo slot direttamente sul **tipo** (`type(a).__len__`). E' cio' che permette a CPython di usare gli slot scritti in C (`tp_as_sequence->sq_length`) e di non pagare una ricerca per attributo a ogni operazione. La ricerca *esplicita* per attributo segue invece la regola normale: istanza, poi tipo, poi MRO.
Conseguenza per i proxy: `__getattr__` non viene *mai* consultato per le operazioni implicite (`len(p)`, `p + q`, `with p:`, `p[0]`, `iter(p)`...), perche' la ricerca non passa dal `__getattribute__` dell'istanza. Per questo `unittest.mock.MagicMock` deve pre-definire tutti i dunder *sul tipo*, e i proxy davvero trasparenti si costruiscono generando dinamicamente una classe.

Soluzione C3 — Name mangling

Stampa `A helper`.
E' il **name mangling**: dentro il corpo di `A`, il compilatore riscrive `self.__helper` in `self._A__helper`; `B` definisce invece `_B__helper`. I due nomi non collidono, quindi non c'e' nessun override.
E' l'unico modo idiomatico in Python per ottenere un metodo **effettivamente non virtuale**: serve per implementare NVI, e soprattutto per evitare collisioni accidentali di nomi nei *mixin* e nelle gerarchie profonde. Non e' un meccanismo di sicurezza: `b._A__helper()` funziona benissimo. Con un solo underscore (`_helper`) non c'e' mangling: e' solo una convenzione, e l'override avviene normalmente.

Soluzione C4 — Dove vive davvero un metodo

(1) `B.f` — dispatch normale lungo `type(b).__mro__`.

(2) `A.f` — chiamata esplicita dell'implementazione della base, equivalente a `super(B, b).f()`. In Python

3 `A.f` e' una funzione semplice, quindi `self` va passato a mano.

(3) `A.f` — le classi sono oggetti mutabili: si e' sostituito il metodo a runtime (*monkey patching*). Nessun altro linguaggio del corso lo permette senza strumenti esterni.

(4) `inst` — l'attributo di istanza vince (e la lambda non riceve `self`: e' una funzione qualunque, non un metodo).

(5) `A.f` — riassegnare `__class__` cambia il tipo dinamico dell'oggetto e quindi il dispatch.

Il motivo del punto (4) e' il **protocollo dei descrittori**: le funzioni sono descrittori *non-data* (definiscono solo `__get__`), e per questi il `__dict__` dell'istanza ha la precedenza. Una `property` e' invece un descrittore *data* (definisce `__get__` *e* `__set__`), e i descrittori data vincono *sempre* sul dizionario dell'istanza: e' proprio cosi' che una property riesce a intercettare le assegnazioni.

Soluzione C5 — Priorita' delle operazioni riflesse

Stampa `B.__radd__`.
Regola generale: per `x + y` Python prova `type(x).__add__` e poi `type(y).__radd__`. **Ma** se `type(y)` e' una *sottoclasse propria* di `type(x)` *e* fornisce un `__radd__` diverso da quello (eventuale) di `type(x)`, il metodo riflesso viene provato **per primo**. In CPython il controllo e' la funzione `method_is_overloaded`: se il tipo di sinistra non ha affatto `__radd__`, quello della sottoclasse conta come "overloaded" e vince.
La motivazione di design e' che una sottoclasse deve sempre poter avere l'ultima parola sulle operazioni che la coinvolgono, altrimenti non potrebbe raffinare il comportamento ereditato (si pensi a `int` e a una sottoclasse che rappresenta unita' di misura).

Soluzione C6 — Il costruttore virtuale che il C++ non ha

Stampa `B`.
In un `classmethod`, `cls` e' legato al tipo su cui e' avvenuta *effettivamente* la chiamata (`B`), quindi `cls()` costruisce una `B`: i classmethod sono **polimorfici**. E' letteralmente il *costruttore virtuale* che in C++ va emulato con `clone()` (A8a) e che in Java richiede una factory esplicita o la reflection.
Con uno `@staticmethod` che fa `return A()` si perde l'informazione sul tipo: la factory restituirebbe sempre una `A` e ogni sottoclasse sarebbe costretta a riscriverla. Vale anche l'inverso: dentro un classmethod, `super().create()` funziona e propaga correttamente `cls`.

Soluzione C7 — Metodi astratti che non astraggono

`Impl()` non solleva nulla perche' il controllo dei metodi astratti non e' fatto dal decoratore, ma da `ABCMeta` in fase di creazione della classe: e' la metaclasse a popolare l'insieme `__abstractmethods__`, che `object.__new__` poi consulta. Il decoratore `@abstractmethod` da solo si limita a impostare `__isabstractmethod__ = True` sulla funzione, e senza `ABCMeta` nessuno legge quel flag.
Correzione: `class Base(abc.ABC):` oppure `class Base(metaclass=abc.ABCMeta):`.
Bonus. `__init_subclass__` e' un hook *per sottoclasse*, invocato alla creazione di ogni classe figlia: e' il modo leggero per validare o registrare le sottoclassi senza scrivere una metaclasse. Una **metaclasse** controlla la creazione dell'oggetto-classe stesso (e puo' provocare conflitti di metaclasse nell'ereditarieta' multipla). `__subclasshook__` interviene invece su `issubclass` e `isinstance` e serve a implementare *duck typing strutturale*: e' cosi' che `collections.abc.Iterable` riconosce come sottoclasse qualunque tipo definisca `__iter__`, senza ereditarieta' esplicita.

7.4 Soluzioni C

Soluzione D1 — Dispatch dinamico scritto a mano

```c
typedef struct Shape Shape;

typedef struct {                    /* la vtable: una per CLASSE */
    void   (*draw)(const Shape *self);
    double (*area)(const Shape *self);
    void   (*dtor)(Shape *self);    /* distruttore virtuale */
    const char *type_name;          /* poor man's RTTI */
} ShapeVT;

struct Shape { const ShapeVT *vptr; };              /* "classe base" */
typedef struct { Shape base; double r; } Circle;    /* "classe derivata" */

static void   circle_draw(const Shape *s)
{ printf("Circle r=%g\n", ((const Circle *)s)->r); }

static double circle_area(const Shape *s)
{ double r = ((const Circle *)s)->r; return 3.14159265 * r * r; }

static void   circle_dtor(Shape *s) { free(s); }

static const ShapeVT CIRCLE_VT = { circle_draw, circle_area, circle_dtor, "Circle" };

Circle *circle_new(double r) {
    Circle *c = malloc(sizeof *c);
    c->base.vptr = &CIRCLE_VT;      /* <-- il "costruttore" installa il vptr */
    c->r = r;
    return c;
}

void draw_all(Shape *const *v, size_t n) {
    for (size_t i = 0; i < n; i++)
        v[i]->vptr->draw(v[i]);     /* == v[i]->draw() del C++ */
}
```

Primo membro. Lo standard garantisce che una struct non abbia padding *all'inizio*, quindi `(Shape *)circle == &circle->base`: la conversione derivata/base e' un no-op e il `this` non va aggiustato. E' esattamente il caso dell'ereditarieta' singola in C++, quello senza thunk (A5).
Il vptr lo inizializzi tu. Se dimentichi quella riga hai un puntatore indeterminato e la prima chiamata virtuale salta nel vuoto. In C++ e' il compilatore a emettere l'assegnamento all'inizio di *ogni* costruttore, e a riscriverlo a ogni livello della gerarchia (che e' poi il motivo di A1).
Distruttore virtuale = una entry `dtor` nella vtable, invocata sempre come `s->vptr->dtor(s)`; ogni "derivata" libera i propri membri e poi chiama esplicitamente il distruttore della base. `dynamic_cast` = confronto del `vptr` (o del tag `type_name`) con quello atteso; per gerarchie a piu' livelli si mette nella vtable un puntatore `parent` e si risale la catena.

Soluzione D2 — Il cast che sembra innocuo

`Circle *` e `Shape *` sono tipi di puntatore **non compatibili**: la conversione `Circle **` → `Shape **` non e' mai implicita e, una volta forzata col cast, l'accesso a `v[i]` legge un oggetto il cui *effective type* e' `Circle *` attraverso un lvalue di tipo `Shape *`. E' una violazione della regola di *aliasing* (*C11 6.5p7*), quindi undefined behavior — innocuo nella pratica perche' tutti i puntatori a struct hanno la stessa rappresentazione, ma il compilatore e' autorizzato a riordinare o eliminare gli accessi.
La garanzia "indirizzo del primo membro = indirizzo della struct" (*6.7.2.1p15*) riguarda **l'indirizzo dell'oggetto**, non la compatibilita' fra *tipi di puntatore*: dice che il cast `Circle *` → `Shape *` da' l'indirizzo giusto, non che un array di `Circle *` *sia* un array di `Shape *`. La regola della *common initial sequence* (*6.5.2.3*) e' poi enunciata solo per struct che condividono una `union` visibile nella stessa unita' di traduzione.
Soluzione corretta: costruire un vero array di `Shape *`, convertendo elemento per elemento (`arr[i] = &c[i]->base;`). La conversione va fatta *sull'oggetto*, non sul puntatore a puntatore — ed e' lo stesso motivo

per cui in Java gli array covarianti producono `ArrayStoreException` (E3).

Soluzione D3 — Traduzione letterale di virtual

- `this` diventa il primo parametro esplicito `Shape *self`;
- `const` diventa `const Shape *self`: nessuna differenza a runtime, solo type checking a compile time;
- la chiamata `s->area()` diventa `s->vptr->area(s)`: due dereferenziazioni (vptr, poi lo slot) piu' una chiamata indiretta;
- la **vtable e' un oggetto statico unico per classe** (`static const ShapeVT`), non per istanza. L'"ereditarieta"' della vtable e' letteralmente: copiare la tabella della base e sovrascrivere gli slot ridefiniti, *mantenendo lo stesso indice* per lo stesso metodo. E' questa invarianza dell'indice a rendere il dispatch $O(1)$ e indipendente dalla profondita' della gerarchia.

Costo: un puntatore per *oggetto* (8 byte su architettura a 64 bit, piu' l'eventuale allineamento) e una tabella di N puntatori per *classe*. Nessun altro overhead: e' il *zero-overhead principle* — paghi solo se usi `virtual`, ed e' il motivo per cui in C++ il default e' non virtuale (E2).

Soluzione D4 — Due polimorfismi diversi

`struct file_operations` e' **overriding**: e' una vtable a tutti gli effetti, condivisa da tutte le istanze di un "tipo" di file (ext4, procfs, un driver di periferica), raggiunta da `struct file` attraverso il campo `f_op`. Ogni filesystem ridefinisce solo le entry che gli servono e lascia `NULL` le altre (che il VFS interpreta come "comportamento di default": un modo per emulare i metodi non astratti della base).
Il comparatore di `qsort` e' invece una **strategy**/callback: la funzione e' passata per parametro alla singola chiamata e non e' associata al *tipo* degli oggetti ordinati; non esiste alcuna entita' "classe" che la erediti. E' il polimorfismo della *Strategy* del GoF, non quello del *Template Method*.
`container_of(ptr, type, member)` risale dall'indirizzo di un membro all'indirizzo della struct che lo contiene, sottraendo `offsetof`. E' il **downcast** da base a derivata (`static_cast<Derived *>(pBase)`) nel caso in cui il sotto-oggetto base *non* sia il primo membro: la stessa aritmetica di offset che genererebbe il compilatore C++.

Soluzione D5 — Perche' il C non puo' avere overriding automatico

Perche' in C ogni chiamata di funzione e' **early binding**: il nome viene risolto dal linker a *un* singolo simbolo, e non esiste alcun canale — ne' un vptr implicito ne' metadati di tipo — che leghi un *oggetto* a un insieme di funzioni. `static inline` negli header duplica la definizione in ogni unita' di traduzione, ma resta comunque una risoluzione statica per nome.
Si aggiungono due impedimenti strutturali:

- senza *name mangling* non potresti nemmeno avere due funzioni `area` con lo stesso nome e parametri diversi nello stesso programma (niente overload, quindi a maggior ragione niente override);
- senza costruttori non esiste un punto garantito in cui installare il vptr: ogni schema di dispatch dinamico va costruito *e mantenuto* a mano dal programmatore.

Ed e' precisamente quello che fanno i grandi progetti C: GObject/GTK, il VFS del kernel Linux, COM su Windows. Il C non e' "incapace" di polimorfismo: si limita a non automatizzarlo.

7.5 Soluzioni trasversali

Soluzione E1 — Tabella comparativa del dispatch

	C++	Java	Python	C
Meccanismo	vtable per classe + vptr per oggetto; indice noto a compile time	`invokevirtual` (vtable) / `invokeinterface` (itable); JIT con inline cache e CHA	lookup nel `__dict__` lungo l'MRO del tipo, con *type method cache*, poi protocollo dei descrittori	manuale: struct di puntatori a funzione
Costo	2 load + chiamata indiretta; devirtualizzabile con `final`/LTO	come C++, ma spesso *inlinato per speculazione* e deottimizzato se serve	il piu' alto: lookup hash + creazione del bound method (mitigato da cache e `LOAD_METHOD`)	identico al C++: e' la stessa cosa scritta a mano
Metodi a runtime	no: layout fissato a compile time	no per una classe gia' caricata (solo agent, redefinition, proxy dinamici)	**si', sempre**: classi mutabili, `__class__` riassegnabile	si', se la vtable non e' `const`
Method hiding	si': name hiding degli overload, campi, membri non virtuali	si': campi, `static`, `private`	non esiste hiding, solo shadowing istanza/classe/MRO	non applicabile
Chiamare la base	`Base::f()` (chiamata non virtuale)	`super.f()` → `invokespecial`; `B.super.f()` per i default	`super().f()` (segue l'MRO!) oppure `A.f(self)`	`parent_vt->f(self)` esplicito

Soluzione E2 — Virtuale per default oppure no

Il C++ segue lo **zero-overhead principle**: chi non usa il polimorfismo non deve pagare ne' il vptr (che oltretutto romperebbe la *layout compatibility* con il C e la natura POD dei tipi) ne' la chiamata indiretta. Per questo `virtual` e' opt-in. Il prezzo e' che rendere virtuale un metodo in un secondo momento e' un **cambio di ABI**, e che dimenticare `virtual ~Base()` e' undefined behavior.
Java sceglie il default virtuale per uniformita' e perche' il costo e' recuperabile *a runtime*: la JVM esegue **Class Hierarchy Analysis** (se una sola classe caricata implementa il metodo, la chiamata viene devirtualizzata e inlinata, con una *guard* e una deottimizzazione se in seguito viene caricata una seconda implementazione) e usa **inline cache** monomorfiche o bimorfiche. E' un'ottimizzazione che il C++ puo' fare solo con LTO o con `final`, perche' la compilazione e' separata e l'insieme delle classi non e' chiuso.
Il **fragile base class problem** taglia in entrambe le direzioni: con il default virtuale ogni metodo pubblico diventa un punto di estensione non intenzionale, e una modifica interna alla base puo' rompere sottoclassi che ne fanno l'override (il caso classico e' `HashSet.addAll` che internamente chiama `add`); con il default non virtuale il rischio si sposta sull'*hiding* silenzioso, che il compilatore non segnala. Python porta il compromesso all'estremo opposto: massima flessibilita' (monkey patching, mixin cooperativi, classi mutabili), nessuna garanzia statica e il costo di dispatch piu' alto dei quattro.

Soluzione E3 — Covarianza e controvarianza

In nessuno dei quattro linguaggi `process(Derived)` sostituisce `process(Base)` in modo corretto, ma per ragioni diverse.
In C++ e Java e' un **overload** con firma diversa: la derivata eredita comunque `process(Base)` e vi aggiunge un secondo metodo (in Java `@Override` darebbe errore; in C++ scatta anche il name hiding, A3). In Python la firma non fa parte dell'identita' del metodo, quindi `def process(self, d)` *sostituisce* davvero quella della base: li' l'override avviene, ma **viola LSP** e i type checker (mypy, pyright) lo segnalano. In C il concetto non esiste.
Il principio di sostituzione di Liskov richiede parametri **controvarianti** (la derivata deve accettare *almeno* tutto cio' che accetta la base) e return **covarianti** (deve promettere *almeno* quanto la base). Restringere il parametro a `Derived` e' covarianza dei parametri, e non e' type-safe: con `Base b = new Derived(); b.process(new Base());` il metodo riceverebbe un oggetto che non sa trattare. E' lo stesso buco degli array covarianti di Java (`ArrayStoreException`) e dei parametri covarianti di Eiffel, entrambi riconosciuti come errori di progetto.
La controvarianza dei parametri sarebbe invece *sicura*, ma quasi nessun linguaggio la implementa perche'

complica enormemente l'overload resolution e nella pratica serve raramente: Python e TypeScript la accettano solo a livello di type checker, Scala e Kotlin la esprimono sui generici (`-T`, `in T`) invece che sui parametri dei metodi.

Soluzione E4 — Sette termini da non confondere

- **Overriding** — ridefinizione, in una sottoclasse, di un metodo con la *stessa firma*, risolta sul tipo *dinamico*. C++ (solo se `virtual`), Java (default), Python (sempre). In C solo emulato a mano.
- **Overloading** — piu' funzioni con lo stesso nome e parametri diversi, risolte a *tempo di compilazione*. C++ e Java. In Python **no**: l'ultima definizione vince, si emula con `functools.singledispatch`. In C no.
- **Hiding / shadowing** — un nome in uno scope piu' interno oscura quello esterno, *senza* polimorfismo: campi e metodi `static` in Java, name hiding degli overload e membri non virtuali in C++. In Python esiste solo lo shadowing istanza/classe/MRO.
- **Dominanza** — solo C++, con ereditarieta' virtuale: un membro di una classe piu' derivata domina quello della base virtuale e risolve un'ambiguita' che altrimenti sarebbe tale (A4).
- **Bridge method** — solo Java: metodo sintetico generato dal compilatore per riconciliare l'erasure dei generici e i return covarianti con la firma che la JVM richiede per il dispatch (B5).
- **Thunk** — C++ (e implementazioni C manuali): trampolino che aggiusta `this` nell'ereditarieta' multipla o virtuale prima di saltare all'override (A5).
- **Final overrider** — C++: per ogni funzione virtuale e ogni classe, l'unica implementazione effettivamente installata in vtable; se non e' unica il programma e' ill-formed. Il concetto esiste anche in Java, ma e' sempre banale per via dell'ereditarieta' singola.

Chapter 8

Ripasso lampo

Le dieci affermazioni da avere in testa entrando all'esame.

1. In C++ il tipo dinamico *cresce* durante la costruzione e *decresce* durante la distruzione: virtuali chiamate da ctor/dtor risolvono alla classe corrente. In Java il tipo dinamico e' quello finale fin dall'inizio, per questo la stessa situazione produce campi non ancora inizializzati.
2. Overload = statico, override = dinamico. Se cambia la firma, non e' un override: e' un overload (e in C++ nasconde pure gli altri).
3. Sono risolti sul **tipo statico**: campi, metodi `static`, argomenti di default, controllo di accesso, scelta dell'overload, aritmetica dei puntatori negli array. Sul tipo dinamico c'e' solo il *corpo* del metodo virtuale.
4. Il controllo di accesso e' ortogonale al dispatch: un `private virtual` si puo' sovrascrivere ma non chiamare (NVI).
5. Distruttore virtuale ogni volta che si distrugge tramite puntatore alla base; ma `delete[]` polimorfico resta UB comunque.
6. Base non virtuale = incorporata, una copia per ramo, offset costante. Base virtuale = condivisa, una copia sola, offset letto a runtime: percio' la costruisce la classe *most derived* e percio' `static_cast` dalla base virtuale alla derivata e' vietato.
7. In Java i generici sono cancellati: i *bridge method* sono la toppa, e l'*heap pollution* il suo effetto collaterale.
8. Nel diamante dei default method vince la classe (*class wins*); fra interfacce non correlate non vince nessuno e si deve fare l'override esplicito con `X.super.f()`.
9. In Python `super()` significa "il prossimo nell'MRO dell'oggetto", non "la superclasse"; i dunder impliciti si cercano sul tipo, mai sull'istanza; il doppio underscore fa mangling e produce metodi di fatto non virtuali.
10. In C il polimorfismo si scrive a mano: struct di puntatori a funzione come vtable, sotto-oggetto base come primo membro, vptr installato dal costruttore, `container_of` al posto del downcast.
11. Liskov: parametri controvarianti (o invarianti), return covarianti, precondizioni non piu' forti, postcondizioni non piu' deboli. Ogni override che viola questo schema e' un bug in attesa.